

Microsoft 365 Copilot — why the GitHub Copilot interception approach doesn't transfer

Written 2026-09-21, as a follow-up to [copilot-desktop-apps-coverage-gap.html](#) 's disambiguation of the word "Copilot." That doc ruled Microsoft 365 Copilot **out of scope permanently** for a simple reason: it's a different company's product, talking to a different backend, with no shared infrastructure to intercept via anything built for GitHub Copilot. This doc answers a narrower, separate question raised afterward: **forget GitHub entirely — could the same *kind of approach* (a local listener a client is redirected to, relaying to the real backend) work against Microsoft 365 Copilot on its own terms?**

Short answer: no, for two independent reasons, checked against Microsoft's own current admin documentation, not guessed. Either one alone would be enough to close this; both being true makes it a firmer close than the VS Code Copilot Chat case, which at least had one real, if gated, lever to chase.

What Microsoft 365 Copilot actually is

Microsoft's own public description: an AI assistant integrated into Word, Excel, PowerPoint, Outlook, Teams, Loop, and Whiteboard. A user's prompt is grounded in their actual organizational data via **Microsoft Graph**, then sent to a large language model on **Azure OpenAI Service**, with the response grounded back into the document/app context. Confirmed distinct from GitHub Copilot on Microsoft's own page: *"a completely separate product from GitHub Copilot."* Structurally the same shape as GitHub Copilot — client app → orchestration/auth layer → LLM backend → response — but a completely different, Microsoft-owned backend with zero relationship to GitHub's CAPI.

Reason 1: there is no redirect lever — the endpoint is fixed, not configurable

GitHub Copilot's VS Code integration had [github-enterprise.uri](#): a real [settings.json](#) key that (for an enterprise-flagged account) determines where the client sends its discovery call, and whose response the client trusts unconditionally for where to send everything after. That's the lever the whole interception mechanism was built on.

Microsoft 365 Copilot has no equivalent. Per Microsoft's own admin documentation ([App and network requirements for Microsoft Copilot admins](#)):

- Copilot's enterprise experiences connect to a fixed set of domains: `copilot.cloud.microsoft`, more broadly `*.cloud.microsoft`, and `*.office.com`. These are the same for every tenant — not resolved from a per-tenant or per-device setting the way `github-enterprise.uri` is.
- Microsoft's explicit guidance to admins is to allow the **entire** `*.cloud.microsoft` domain, stating directly: "*Microsoft doesn't support allowing partial or only selected Microsoft 365 application URLs within the `*.cloud.microsoft` domain.*" That's a statement about network policy, but it doubles as confirmation there's no narrower, single addressable Copilot-specific endpoint an admin (or a config write) could isolate and redirect — the whole wildcard domain is the unit Microsoft expects you to treat atomically.
- No config file, registry key, environment variable, or admin policy surfaced in this documentation that repoints where any of this traffic goes. Compare this to VS Code, where `github-enterprise.uri` is a real, documented, user-or-workspace-scoped `settings.json` key that does exactly that (for the one account type that reads it).

Consequence: there's no equivalent of "write one config key, watch the client connect to us instead." Nothing here determines its own destination from anything write-accessible. This alone is sufficient to close the question via the app-config route — but a config key isn't the only way to redirect a connection, so the network-layer alternative below was worked through too, before concluding it doesn't actually get around this either.

A different path to Reason 1: DNS-wildcard redirect + dynamic cert minting

Considered as an alternative once the app-config route above closed: instead of asking the *application* to redirect itself, redirect at the *network* layer, the same way the resident daemon already can for other agents.

A hosts file can't do this — wildcards aren't representable there. `/etc/hosts` (and its Windows equivalent) only matches exact hostnames, one line per FQDN. Since Microsoft explicitly won't publish the real underlying FQDNs behind `*.cloud.microsoft` / `*.office.com` (the same "hyperscale and dynamic" reasoning quoted above), there's no static list to enumerate and redirect. What this actually needs is a **local DNS resolver that performs real wildcard matching** — intercepting any query ending in the relevant suffixes and answering with the listener's own address instead of the real one. Mechanically doable from a privileged daemon; not a new category of capability.

DNS redirection alone produces a connection, not a working one. The client still expects a certificate for whatever real hostname it thinks it dialed. Closing that gap needs the same CA-trust mechanism already used elsewhere in this codebase for other agents: mint a leaf certificate for whatever hostname/SNI the client actually requested, on the fly, signed by a root CA already installed and trusted on the device — the same technique `mitmproxy` / Burp use for arbitrary-host interception, not a fixed pre-issued cert. Reason 2 found no evidence of certificate pinning on this

traffic specifically, so this would likely pass ordinary validation — *likely*, not confirmed; pinning wasn't ruled out with certainty either.

The real new cost this introduces: the blast radius is far larger than Copilot.

`*.cloud.microsoft` and `*.office.com` aren't Copilot-specific — SharePoint, OneDrive, Teams, and Outlook itself share them. A wildcard DNS redirect catches all of it. Making this Copilot-only would require the listener to inspect the SNI/Host per connection and decide, live, which hostnames to actually terminate-and-relay versus which to pass through untouched — and Microsoft's own guidance says the opposite of that: "*Microsoft doesn't support allowing partial or only selected...URLs within `*.cloud.microsoft` ... allow the entire domain.*" Selective interception inside that wildcard is close to the exact shape of thing their docs already warn causes the Reason 2 failures — so this path to closing Reason 1 makes Reason 2 *more* likely to bite, not less.

And it only ever closes Reason 1. Landing the connection on the listener doesn't touch the SignalR relay problem underneath it — Design A/B below would still be the only two shapes available for what to actually do with the traffic once it arrives.

Reason 2: the traffic is a persistent SignalR WebSocket, not request/response HTTP — and Microsoft documents interception breaking it

Every interception mechanism this codebase has actually built — `kinds::copilot_discovery`'s direct call-and-rewrite, the `forward()` /Optimus tunnel every `BaseUrl` agent uses — is shaped around discrete HTTP request/response pairs: a request comes in, gets relayed or answered, a response goes back. Microsoft 365 Copilot's traffic is not that shape, and this is now confirmed at the protocol level, not just at the network-connectivity level.

Official Microsoft documentation confirms the connectivity requirement. Per the admin doc's **WebSockets (WSS) protocol requirements** section:

Verify that your network supports full WSS connectivity from user devices running Microsoft 365 applications to the following domains: Microsoft Copilot enterprise experiences:
`*.office.com`, `*.cloud.microsoft` and `copilot.cloud.microsoft`.

And, listed as a typical cause of "*Copilot application failures*":

Network devices attempting to perform Transport Layer Security (TLS) inspection of connections

Independent reverse-engineering confirms the actual protocol, not just that *some* WebSocket is involved. A third-party project (`cramt/m365-copilot-proxy`) built a working proxy for the built-in BizChat/Office-web Copilot experience specifically (Word/Excel/etc., not consumer

Bing Copilot — the two are architecturally distinct and easy to conflate; an earlier pass of this research briefly did), which required capturing and matching the live wire protocol, not just guessing at it:

- **Transport:** SignalR-over-WebSocket, at `wss://substrate.office.com/m365Copilot/Chathub/{oid}@{tid}` — the auth token rides in the URL query string, not a header.
- **Handshake:** a JSON protocol-negotiation frame (`{"protocol":"json","version":1}`), then typed frames (invocation, stream item, completion, ping, close) separated by `0x1E` record-separator bytes — genuine, verifiable SignalR wire-protocol detail.
- **Keep-alive:** the client must answer server ping frames or the connection dies — a real, bidirectional, stateful protocol, not a one-way stream.

A related doc on Copilot's newer "Cowork" agentic feature ([Cowork network endpoints](#)) is direct about the failure mode a naive interceptor hits:

SSL/TLS inspection isn't supported and might interrupt long-lived SSE connections, which can result in message delivery failures, retries, or frozen tasks.

What this does and doesn't prove. Microsoft's language describes what happens with *generic corporate TLS-inspecting proxy appliances* — hardware typically tuned for short-lived HTTP request/response, which often mishandle long-lived, stateful connections (aggressive idle timeouts, buffering that breaks keep-alives, no awareness of SignalR's own ping/pong contract). It is not a claim that no relay could ever handle this traffic — a purpose-built proxy that understands the SignalR framing, holds the connection open, and answers pings correctly is a solved class of problem in general (this is how API gateways and service meshes handle WebSocket/SignalR traffic routinely). No evidence of application-level certificate pinning (an absolute technical block, distinct from a reliability complaint) was found in this pass — the language throughout is about connection fragility under naive inspection, not a stated refusal to accept a foreign certificate.

A genuinely open, unresolved wrinkle: SignalR's standard client behavior is to `negotiate` over plain HTTP first, receive a list of transports the server supports, and *fall back* to Server-Sent-Events or long-polling when a WebSocket upgrade fails — this exists specifically for networks that block WS upgrades, which is exactly this scenario. The reverse-engineering above didn't observe or document that fallback in practice, so it's unconfirmed whether Microsoft's Chathub deployment actually honors a downgrade or is configured WebSocket-only. If it does downgrade, the interception question changes shape entirely: forcing the WS upgrade to fail might push the client onto a transport (HTTP long-polling) that existing request/response relay infrastructure already knows how to handle, without needing to build a new SignalR-aware WebSocket relay at all. Unconfirmed either way — a real next step if this is ever revisited, not something to treat as settled.

What remains genuinely unverified: whether this codebase's own relay infrastructure (`forward()` /Optimus, or a hypothetical new listener kind) is shaped to proxy a persistent SignalR WebSocket (or, per the wrinkle above, a forced-fallback long-polling transport) without the failure modes Microsoft describes. Everything built so far is request/response-shaped. Either path would be new, real engineering work, not a reuse of anything that exists today — and it would only be worth attempting after Reason 1 is somehow solved, which it currently isn't.

Considered and rejected: bridge WSS↔HTTP around Optimus's HTTP-only relay

A natural next idea, raised and worked through in this investigation: since Optimus's `forward()` only ever speaks HTTP, could our own listener terminate the client's WebSocket, translate each frame into something Optimus *can* relay, and translate the response back into WS frames on the way out — the same "different shape on each side of the listener" pattern the `CopilotDiscoveryRelay` mechanism already uses for GitHub Copilot's discovery leg? Worked through fully, this splits into two concrete designs, and both run into a real, structural problem rather than a shallow one.

Design A — bridge to a *different*, genuinely HTTP-shaped backend. Microsoft does expose an official HTTP/SSE surface for Copilot chat: the Graph `chatOverStream` API (§ above). In principle, the listener could translate each client "send message" frame into a POST against that endpoint, relay it through Optimus unchanged, and translate the SSE response back into WS frames.

The problem: **that endpoint is a different product surface**, built for third-party developers writing their own bots against Copilot — not the backend the built-in Word/Excel UI actually talks to. It almost certainly lacks the same document-grounding, citations, and session context the native experience depends on. This wouldn't be *intercepting* the real exchange; it would be silently substituting a different, likely-degraded conversation for it. That defeats the actual point of wanting this visibility in the first place — governing what really happened, not a lookalike generated on the side.

Design B — a true transport bridge that preserves the real exchange. Encode the actual SignalR frames as opaque bytes in an HTTP body, relay that blob through Optimus untouched (Optimus doesn't need to understand SignalR at all here, just move bytes — within what `forward()` already does), and translate back on the way out.

The problem: **something on the far end still has to open a real WebSocket to `substrate.office.com`**. Optimus's `forward()` speaks HTTP to the upstream; it has no "open a WebSocket to the real backend and tunnel these bytes through it" capability. This design doesn't eliminate the WebSocket-handling problem — it relocates *who* has to solve it, from this codebase's listener to Optimus itself, which isn't code in this repo; it's Zscaler's separate cloud

gateway product. That's a materially bigger ask than anything else in this investigation, all of which stayed inside `ai-protect` / `ai-gateway` .

The sharper version of Design B, and why it's a tradeoff rather than a loophole. If instead the listener itself opened the real WebSocket to Microsoft directly — bypassing Optimus entirely, the same precedent `kinds::copilot_discovery` already sets for GitHub Copilot's discovery leg — that's mechanically buildable with no new dependency on Optimus. But the discovery leg could bypass Optimus safely only because it's low-stakes account metadata, nothing worth policy-inspecting. M365 Copilot's chat content is the opposite: it's exactly the traffic worth inspecting — the actual prompts and completions, the real DLP-relevant material. Bypassing Optimus to make the mechanics work would mean building real interception while discarding the one property that made it worth building.

Net: neither design gets past Reason 2 without giving something else up — either the fidelity of what's actually being observed (Design A), or a dependency on new capability in a system this codebase doesn't own (Design B, in its honest form), or the policy-enforcement point of the whole exercise (Design B's bypass variant). And all three still sit behind Reason 1, which none of this touches — this section answers "if we already had the traffic, could we route it through Optimus," not "how do we get the traffic."

Net assessment

	GitHub Copilot (VS Code)	Microsoft 365 Copilot
Redirect lever	<code>github-enterprise.uri</code> — real, but gated by the <code>enterprise:true</code> account flag (§3.1 of the VS Code write-up)	None found — fixed, wildcarded, tenant-wide domain
Traffic shape	HTTP request/response — matches existing relay infrastructure	Persistent SignalR-over-WebSocket (confirmed protocol detail, not just connectivity) — a different shape than anything built so far, with an unconfirmed long-polling fallback that could change this
Documented interception risk	None found for the mechanism itself	Microsoft explicitly documents TLS inspection causing failures on this traffic
Verdict	Mechanism proven correct with real traffic; account-gate is the one open, hard-to-close gap	Closed — no known foothold, and a second, harder engineering problem (or a different, unconfirmed easier one, if the fallback transport is real) sits behind where the first one would have been

Recommendation: don't pursue this further under the current approach. The VS Code Copilot Chat case at least has one concrete, real mechanism with a single named gap (the `enterprise:true` account flag) worth watching for a future closure. Microsoft 365 Copilot has no comparable foothold at all — closing Reason 1 would require Microsoft shipping some new, currently-nonexistent redirect capability, and closing Reason 2 would still require solving a WebSocket-relay engineering problem this codebase has never attempted. Revisit only if Microsoft's own network requirements documentation changes to describe a real, addressable, redirectable endpoint for Copilot traffic specifically.

Sources

- [App and network requirements for Microsoft Copilot admins](#) — Microsoft Learn, updated 2026-09-08 (official)
- [Microsoft Copilot Cowork network endpoints \(Preview\)](#) — Microsoft Support (official)
- [Microsoft 365 URLs and IP address ranges](#) — Microsoft Learn (official)
- [cramt/m365-copilot-proxy](#) — [M365 Copilot API docs](#) — third-party, reverse-engineered against a real working proxy (not Microsoft-published; cited for the specific SignalR/WebSocket protocol detail Microsoft's own docs don't spell out)